SET 3

1. Using C program, Implement and demonstrate the Go-Back-N ARQ protocol for reliable data transmission.

2. Build a network using Packet Tracer with EIGRP (Enhanced Interior Gateway Routing Protocol) configured on 3 routers. Verify the routing table and connectivity.

3. Using Wireshark, Capture and analyze the TCP three-way handshake process between a client and server for different header fields.

4. Design an Extended Star Topology using Packet Tracer with 3 switches and 15 systems, connecting each switch to a central switch.

1.

Aim:

To implement and demonstrate the Go-back-N ARQ protocol using C for reliable data transmission

Procedure

1. Start the C program
2. Enter the number of frames
3. Enter the window size
4. Transmit the frames according to the window size
5. Enter the acknowledgement received
6. If the acknowledgement is correct, move the window forward
7. If an acknowledgement is lost or incorrect, retransmit the frames from the required frame
8. Continue until all frames are successfully transmitted

Coding:

```
#include <stdio.h>
int main()
{
    int n, w, i;
    printf("Enter number of frames:");
    Scanf ("%d", &n);
    printf("Enter windoro Size:");
    Scanf("%d, &w);
    for (i=1 ; i<=n; i++)
    {
```

```
        printf("Sending Frame %d\n", i);
        if (i%w == 0)
            printf("Acknowledgement received\n");
    }
    printf("All frames transmitted succesfully\n");
    return 0;
}
```

Sample Output

```
Enter number of frames: 6
Enter window size: 3
Sending Frame 1
Sending Frame 2
Sending Frame 3
Acknowledgement received
Sending Frame 4
Sending Frame 5
Sending Frame 6
Acknowledgement received
All frames transmitted succesfully
```

Result:

Thus, the Go-Back-N ARQ protocol was succesfully implemented using c for reliable data transmission

Main.c

```
int main() {
    while (base < n) {

        /*
         * If a frame is lost, all frames after it in the
         * current window are retransmitted after timeout.
         */
        if (firstLost != -1) {
            printf("\n*** TIMEOUT for Frame %d ***\n", firstLost);
            printf("Go-Back-N: Retransmitting from Frame %d\n",
                   firstLost);

            base = firstLost;
        }
        else {
            /*
             * All frames in the current window were received.
             * Move the window forward.
             */
            base = nextFrame;
        }

        printf("\n");
    }

    printf("================================\n");
    printf("All %d frames transmitted successfully!\n", n);
    printf("Go-Back-N ARQ transmission completed.\n");
    printf("================================\n");

    return 0;
}
```

Console | I/O | AI Agent | 51.0s

```
---------------------------------
Sending window: 6 7 8 9
  Frame 6 received successfully.
  ACK 6 received by sender.
  Frame 7 received successfully.
  ACK 7 received by sender.
  Frame 8 received successfully.
  ACK 8 received by sender.
  Frame 9 LOST!

*** TIMEOUT for Frame 9 ***
Go-Back-N: Retransmitting from Frame 9

---------------------------------
Sending window: 9
  Frame 9 LOST!

*** TIMEOUT for Frame 9 ***
Go-Back-N: Retransmitting from Frame 9

---------------------------------
Sending window: 9
  Frame 9 received successfully.
  ACK 9 received by sender.

================================
All 10 frames transmitted successfully!
Go-Back-N ARQ transmission completed.
================================
```

Main.c

```
int main() {
    while (base < n) {

        /*
         * If a frame is lost, all frames after it in the
         * current window are retransmitted after timeout.
         */
        if (firstLost != -1) {
            printf("\n*** TIMEOUT for Frame %d ***\n", firstLost);
            printf("Go-Back-N: Retransmitting from Frame %d\n",
                   firstLost);

            base = firstLost;
        }
        else {
            /*
             * All frames in the current window were received.
             * Move the window forward.
             */
            base = nextFrame;
        }

        printf("\n");
    }

    printf("================================\n");
    printf("All %d frames transmitted successfully!\n", n);
    printf("Go-Back-N ARQ transmission completed.\n");
    printf("================================\n");

    return 0;
}
```

Console | I/O | AI Agent

```
Sending window: 6 7 8 9
  Frame 6 LOST!

*** TIMEOUT for Frame 6 ***
Go-Back-N: Retransmitting from Frame 6

---------------------------------
Sending window: 6 7 8 9
  Frame 6 received successfully.
  ACK 6 received by sender.
  Frame 7 received successfully.
  ACK 7 received by sender.
  Frame 8 received successfully.
  ACK 8 received by sender.
  Frame 9 LOST!

*** TIMEOUT for Frame 9 ***
Go-Back-N: Retransmitting from Frame 9

---------------------------------
Sending window: 9
  Frame 9 LOST!

*** TIMEOUT for Frame 9 ***
Go-Back-N: Retransmitting from Frame 9

---------------------------------
Sending window: 9
  Frame 9 received successfully.
```

Main.c

```
int main() {
    while (base < n) {

        /*
         * If a frame is lost, all frames after it in the
         * current window are retransmitted after timeout.
         */
        if (firstLost != -1) {
            printf("\n*** TIMEOUT for Frame %d ***\n", firstLost);
            printf("Go-Back-N: Retransmitting from Frame %d\n",
                   firstLost);

            base = firstLost;
        }
        else {
            /*
             * All frames in the current window were received.
             * Move the window forward.
             */
            base = nextFrame;
        }

        printf("\n");
    }

    printf("================================\n");
    printf("All %d frames transmitted successfully!\n", n);
    printf("Go-Back-N ARQ transmission completed.\n");
    printf("================================\n");

    return 0;
}
```

Console | I/O | AI Agent | 51.0s

```
===== Go-Back-N ARQ Simulation =====
Enter number of frames (max 20): 10

Window Size = 4
Frame Loss Probability = 30%

---------------------------------
Sending window: 0 1 2 3
  Frame 0 received successfully.
  ACK 0 received by sender.
  Frame 1 received successfully.
  ACK 1 received by sender.
  Frame 2 LOST!

*** TIMEOUT for Frame 2 ***
Go-Back-N: Retransmitting from Frame 2

---------------------------------
Sending window: 2 3 4 5
  Frame 2 received successfully.
  ACK 2 received by sender.
  Frame 3 received successfully.
  ACK 3 received by sender.
  Frame 4 received successfully.
  ACK 4 received by sender.
  Frame 5 received successfully.
  ACK 5 received by sender.

---------------------------------
```

Main.c

```
int main() {
    while (base < n) {

        /*
         * If a frame is lost, all frames after it in the
         * current window are retransmitted after timeout.
         */
        if (firstLost != -1) {
            printf("\n*** TIMEOUT for Frame %d ***\n", firstLost);
            printf("Go-Back-N: Retransmitting from Frame %d\n",
                   firstLost);

            base = firstLost;
        }
        else {
            /*
             * All frames in the current window were received.
             * Move the window forward.
             */
            base = nextFrame;
        }

        printf("\n");
    }

    printf("================================\n");
    printf("All %d frames transmitted successfully!\n", n);
    printf("Go-Back-N ARQ transmission completed.\n");
    printf("================================\n");

    return 0;
}
```

Console | I/O | AI Agent | 51.0s

```
===== Go-Back-N ARQ Simulation =====
Enter number of frames (max 20): 10

Window Size = 4
Frame Loss Probability = 30%

---------------------------------
Sending window: 0 1 2 3
  Frame 0 received successfully.
  ACK 0 received by sender.
  Frame 1 received successfully.
  ACK 1 received by sender.
  Frame 2 LOST!

*** TIMEOUT for Frame 2 ***
Go-Back-N: Retransmitting from Frame 2

---------------------------------
Sending window: 2 3 4 5
  Frame 2 received successfully.
  ACK 2 received by sender.
  Frame 3 received successfully.
  ACK 3 received by sender.
  Frame 4 received successfully.
  ACK 4 received by sender.
  Frame 5 received successfully.
  ACK 5 received by sender.

---------------------------------
```

Aim:

To build a network using 3 routers, configure EIGRP, and verify routing table entries and connectivity

Procedure:

1. Open Cisco Packet Tracer
2. Place three routers, three switches and required PCs
3. Connect the routers and switches using suitable cables
4. Assign IP Addresses to the router interfaces and PCs
5. Configure EIGRP on all three routers using the same autonomous system number
6. Advertise the connected networks using the network command
7. Use show ip route to verify the routing table
8. Use show ip eigrp neighbors to verify EIGRP neighbors
9. Use ping to check connectivity between PCs

Sample Output:

R1 # show ip eigrp neighbors

H	Address	Interface
0	10.0.0.2	Gigabit Ethernet 0/1

R1 # show ip route

D 192.168.3.0/24 [90/...] via 10.0.0.2

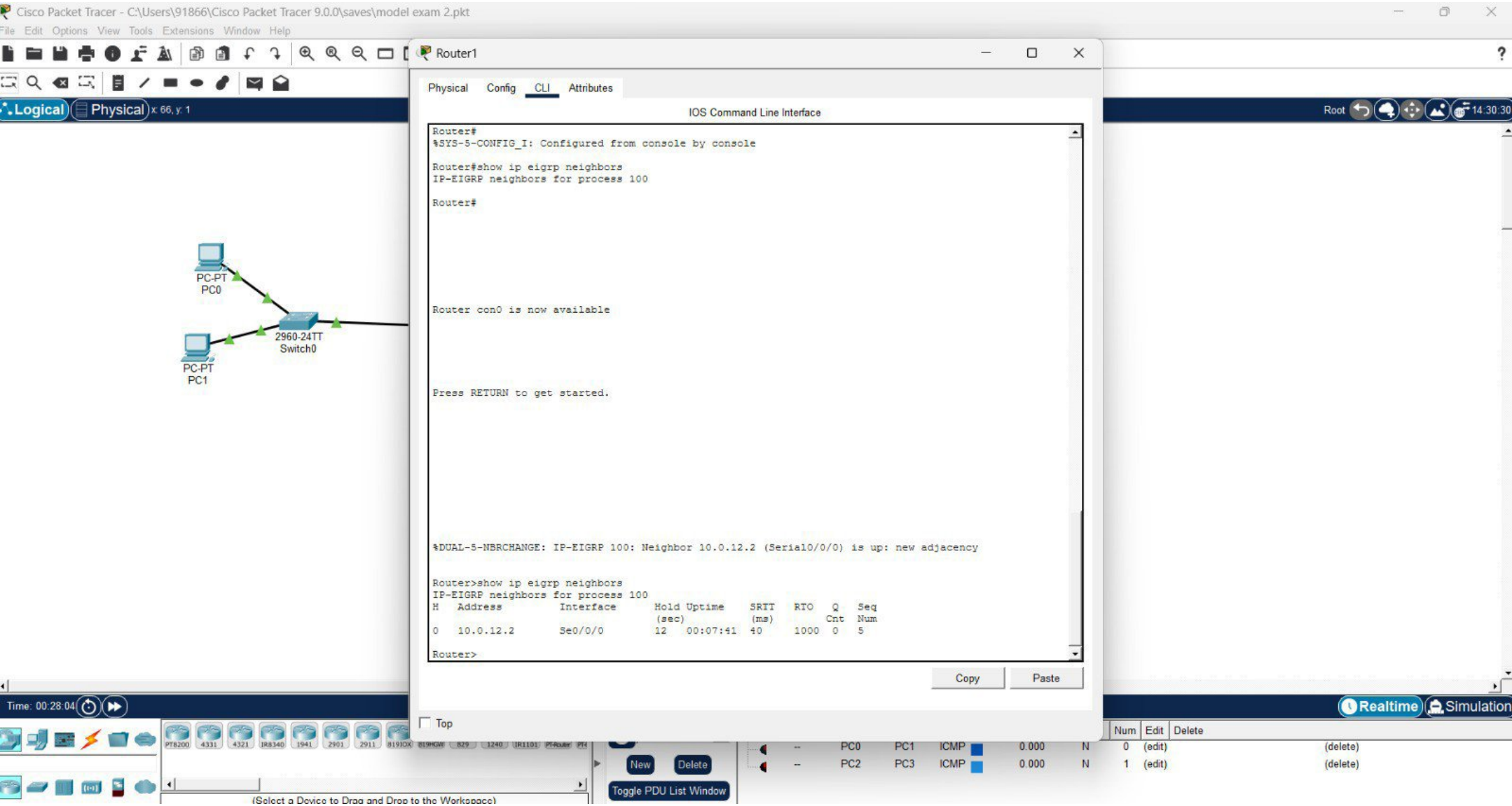
PC1 > ping 192.168.3.10

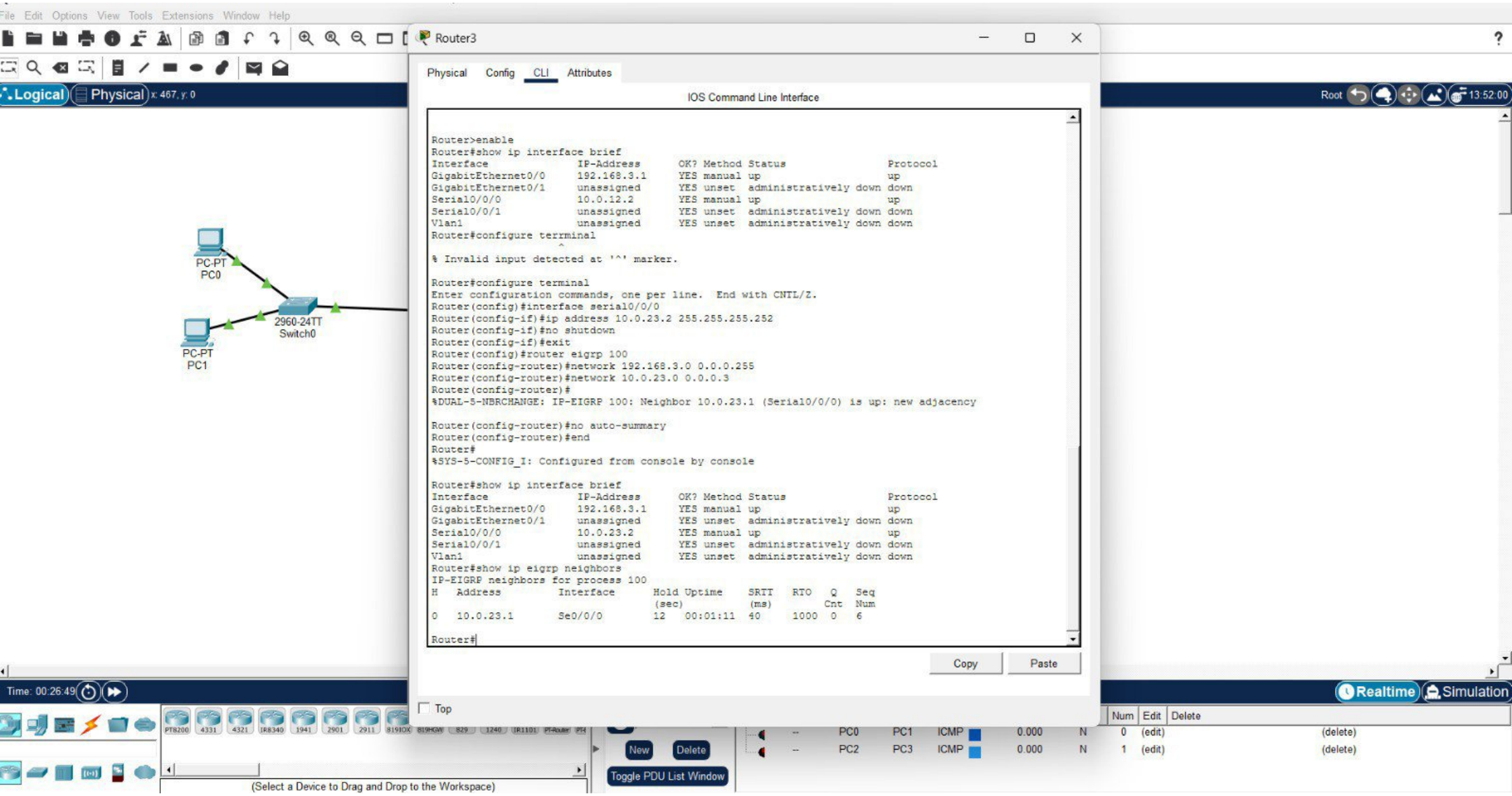
Reply from 192.168.3.10
Reply from 192.168.3.10
Reply from 192.168.3.10

Ping succesful

Result:

Thus, EIGRP was succesfully configured on the three routers and connectivity between the networks was verified.


Cisco Packet Tracer - C:\Users\91866\Cisco Packet Tracer 9.0.0\saves\model exam 2.pkt
File Edit Options View Tools Extensions Window Help
Logical
Physical
x: 66, y: 1
Root
14:30:30
PC-PT
PC0
2960-24TT
Switch0
PC-PT
PC1
Router1
Physical
Config
CLI
Attributes
IOS Command Line Interface
Router#
%SYS-5-CONFIG_I: Configured from console by console
Router#show ip eigrp neighbors
IP-EIGRP neighbors for process 100
Router#
Router con0 is now available
Press RETURN to get started.
%DUAL-5-NBRCHANGE: IP-EIGRP 100: Neighbor 10.0.12.2 (Serial0/0/0) is up: new adjacency
Router>show ip eigrp neighbors
IP-EIGRP neighbors for process 100
H Address Interface Hold Uptime SRTT RTO Q Seq
(sec) (ms) Cnt Num
0 10.0.12.2 Se0/0/0 12 00:07:41 40 1000 0 5
Router>
Copy
Paste
Top
Time: 00:28:04
Realtime
Simulation
New
Delete
Toggle PDU List Window
Num
Edit
Delete
PC0
PC1
ICMP
0.000
N
0
(edit)
(delete)
PC2
PC3
ICMP
0.000
N
1
(edit)
(delete)


File Edit Options View Tools Extensions Window Help
Logical
Physical
x: 467, y: 0
Root
13:52:00
PC-PT
PC0
2960-24TT
Switch0
PC-PT
PC1
Router3
Physical
Config
CLI
Attributes
IOS Command Line Interface
Router>enable
Router#show ip interface brief
Interface IP-Address OK? Method Status Protocol
GigabitEthernet0/0 192.168.3.1 YES manual up up
GigabitEthernet0/1 unassigned YES unset administratively down down
Serial0/0/0 10.0.12.2 YES manual up up
Serial0/0/1 unassigned YES unset administratively down down
Vlan1 unassigned YES unset administratively down down
Router#configure terrminal
^
% Invalid input detected at '^' marker.
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface serial0/0/0
Router(config-if)#ip address 10.0.23.2 255.255.255.252
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#router eigrp 100
Router(config-router)#network 192.168.3.0 0.0.0.255
Router(config-router)#network 10.0.23.0 0.0.0.3
Router(config-router)#
%DUAL-5-NBRCHANGE: IP-EIGRP 100: Neighbor 10.0.23.1 (Serial0/0/0) is up: new adjacency
Router(config-router)#no auto-summary
Router(config-router)#end
Router#
%SYS-5-CONFIG_I: Configured from console by console
Router#show ip interface brief
Interface IP-Address OK? Method Status Protocol
GigabitEthernet0/0 192.168.3.1 YES manual up up
GigabitEthernet0/1 unassigned YES unset administratively down down
Serial0/0/0 10.0.23.2 YES manual up up
Serial0/0/1 unassigned YES unset administratively down down
Vlan1 unassigned YES unset administratively down down
Router#show ip eigrp neighbors
IP-EIGRP neighbors for process 100
H Address Interface Hold Uptime SRTT RTO Q Seq
(sec) (ms) Cnt Num
0 10.0.23.1 Se0/0/0 12 00:01:11 40 1000 0 6
Router#
Copy
Paste
Top
Time: 00:26:49
Realtime
Simulation
(Select a Device to Drag and Drop to the Workspace)
New
Delete
Toggle PDU List Window
Num
Edit
Delete
PC0 PC1 ICMP 0.000 N 0 (edit) (delete)
PC2 PC3 ICMP 0.000 N 1 (edit) (delete)

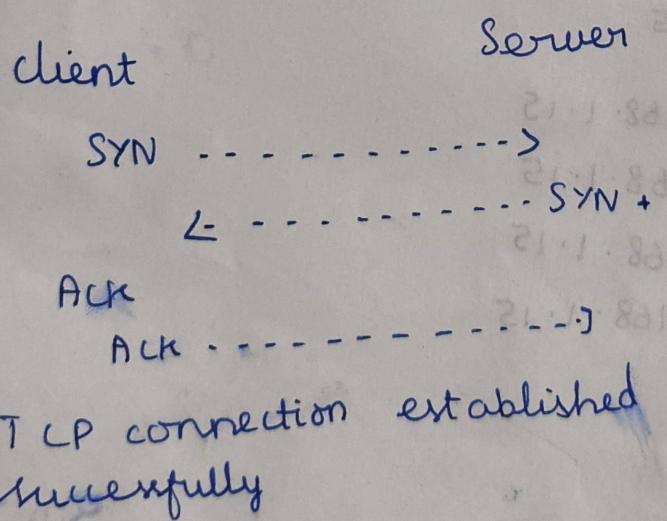
Aim:

To capture and analyze the TCP three-way handshake between a client and server using Wireshark.

Procedure

1. Open Wireshark
2. Select the required network interface
3. Start packet capture
4. Generate TCP traffic between the client and server
5. Stop the capture
6. Apply the filter tcp.
7. Identify the three packets: SYN, SYN-ACK, and ACK
8. Analyze the source port, destination port, sequence number, acknowledgment number, and TCP flags

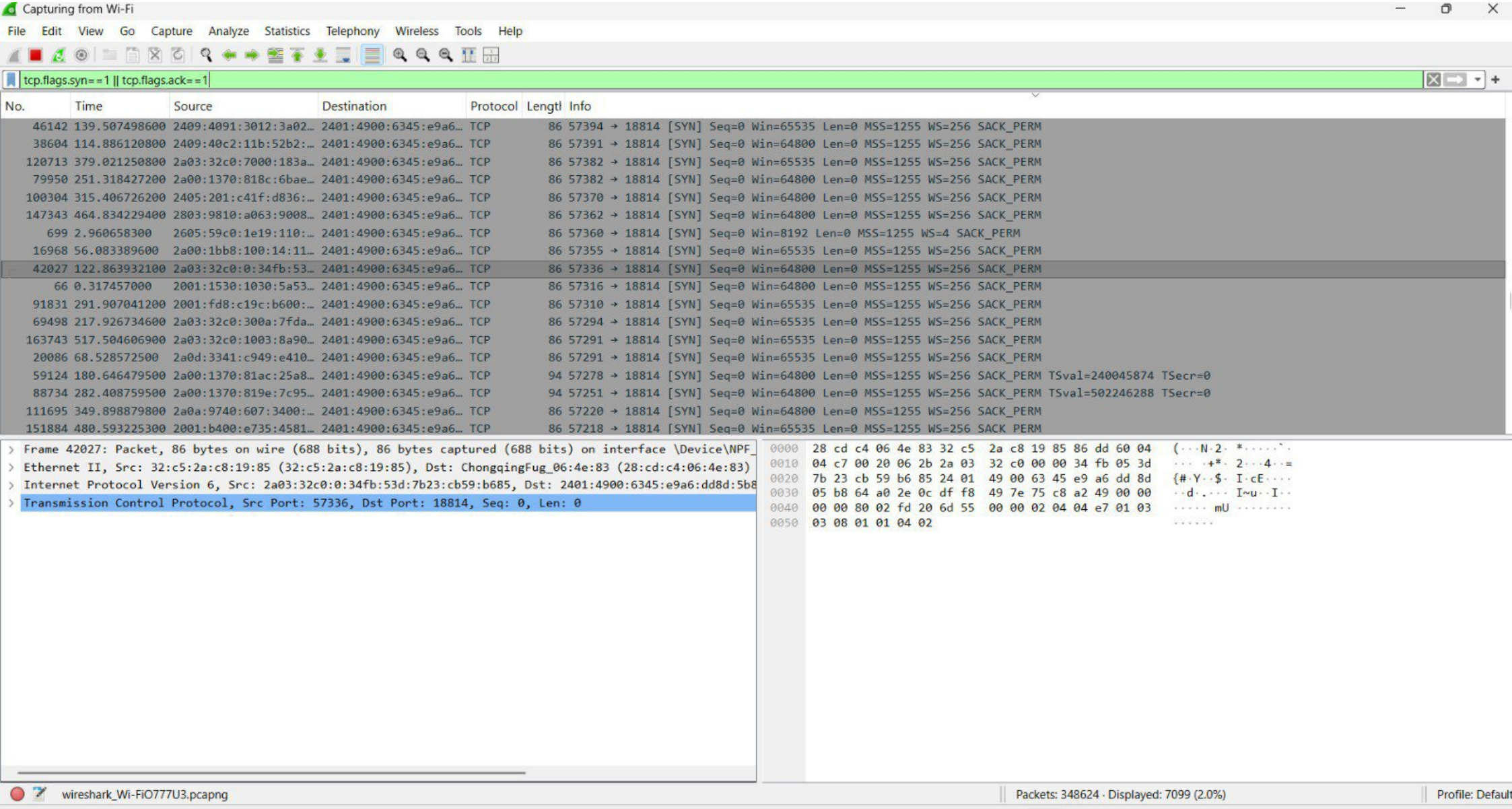
Sample Output:



TCP connection established successfully

Result:

Thus, the TCP three-way handshake was successfully captured and analyzed using Wireshark


Capturing from Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp.flags.syn==1 || tcp.flags.ack==1
No. Time Source Destination Protocol Lengtl Info
46142 139.507498600 2409:4091:3012:3a02… 2401:4900:6345:e9a6… TCP 86 57394 → 18814 [SYN] Seq=0 Win=65535 Len=0 MSS=1255 WS=256 SACK_PERM
38604 114.886120800 2409:40c2:11b:52b2:… 2401:4900:6345:e9a6… TCP 86 57391 → 18814 [SYN] Seq=0 Win=64800 Len=0 MSS=1255 WS=256 SACK_PERM
120713 379.021250800 2a03:32c0:7000:183a… 2401:4900:6345:e9a6… TCP 86 57382 → 18814 [SYN] Seq=0 Win=65535 Len=0 MSS=1255 WS=256 SACK_PERM
79950 251.318427200 2a00:1370:818c:6bae… 2401:4900:6345:e9a6… TCP 86 57382 → 18814 [SYN] Seq=0 Win=64800 Len=0 MSS=1255 WS=256 SACK_PERM
100304 315.406726200 2405:201:c41f:d836:… 2401:4900:6345:e9a6… TCP 86 57370 → 18814 [SYN] Seq=0 Win=64800 Len=0 MSS=1255 WS=256 SACK_PERM
147343 464.834229400 2803:9810:a063:9008… 2401:4900:6345:e9a6… TCP 86 57362 → 18814 [SYN] Seq=0 Win=64800 Len=0 MSS=1255 WS=256 SACK_PERM
699 2.960658300 2605:59c0:1e19:110:… 2401:4900:6345:e9a6… TCP 86 57360 → 18814 [SYN] Seq=0 Win=8192 Len=0 MSS=1255 WS=4 SACK_PERM
16968 56.083389600 2a00:1bb8:100:14:11… 2401:4900:6345:e9a6… TCP 86 57355 → 18814 [SYN] Seq=0 Win=65535 Len=0 MSS=1255 WS=256 SACK_PERM
42027 122.863932100 2a03:32c0:0:34fb:53… 2401:4900:6345:e9a6… TCP 86 57336 → 18814 [SYN] Seq=0 Win=64800 Len=0 MSS=1255 WS=256 SACK_PERM
66 0.317457000 2001:1530:1030:5a53… 2401:4900:6345:e9a6… TCP 86 57316 → 18814 [SYN] Seq=0 Win=64800 Len=0 MSS=1255 WS=256 SACK_PERM
91831 291.907041200 2001:fd8:c19c:b600:… 2401:4900:6345:e9a6… TCP 86 57310 → 18814 [SYN] Seq=0 Win=65535 Len=0 MSS=1255 WS=256 SACK_PERM
69498 217.926734600 2a03:32c0:300a:7fda… 2401:4900:6345:e9a6… TCP 86 57294 → 18814 [SYN] Seq=0 Win=65535 Len=0 MSS=1255 WS=256 SACK_PERM
163743 517.504606900 2a03:32c0:1003:8a90… 2401:4900:6345:e9a6… TCP 86 57291 → 18814 [SYN] Seq=0 Win=65535 Len=0 MSS=1255 WS=256 SACK_PERM
20086 68.528572500 2a0d:3341:c949:e410… 2401:4900:6345:e9a6… TCP 86 57291 → 18814 [SYN] Seq=0 Win=65535 Len=0 MSS=1255 WS=256 SACK_PERM
59124 180.646479500 2a00:1370:81ac:25a8… 2401:4900:6345:e9a6… TCP 94 57278 → 18814 [SYN] Seq=0 Win=64800 Len=0 MSS=1255 WS=256 SACK_PERM TSval=240045874 TSecr=0
88734 282.408759500 2a00:1370:819e:7c95… 2401:4900:6345:e9a6… TCP 94 57251 → 18814 [SYN] Seq=0 Win=64800 Len=0 MSS=1255 WS=256 SACK_PERM TSval=502246288 TSecr=0
111695 349.898879800 2a0a:9740:607:3400:… 2401:4900:6345:e9a6… TCP 86 57220 → 18814 [SYN] Seq=0 Win=64800 Len=0 MSS=1255 WS=256 SACK_PERM
151884 480.593225300 2001:b400:e735:4581… 2401:4900:6345:e9a6… TCP 86 57218 → 18814 [SYN] Seq=0 Win=65535 Len=0 MSS=1255 WS=256 SACK_PERM
Frame 42027: Packet, 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface \Device\NPF_
Ethernet II, Src: 32:c5:2a:c8:19:85 (32:c5:2a:c8:19:85), Dst: ChongqingFug_06:4e:83 (28:cd:c4:06:4e:83)
Internet Protocol Version 6, Src: 2a03:32c0:0:34fb:53d:7b23:cb59:b685, Dst: 2401:4900:6345:e9a6:dd8d:5b8
Transmission Control Protocol, Src Port: 57336, Dst Port: 18814, Seq: 0, Len: 0
0000 28 cd c4 06 4e 83 32 c5 2a c8 19 85 86 dd 60 04
0010 04 c7 00 20 06 2b 2a 03 32 c0 00 00 34 fb 05 3d
0020 7b 23 cb 59 b6 85 24 01 49 00 63 45 e9 a6 dd 8d
0030 05 b8 64 a0 2e 0c df f8 49 7e 75 c8 a2 49 00 00
0040 00 00 80 02 fd 20 6d 55 00 00 02 04 04 e7 01 03
0050 03 08 01 01 04 02
wireshark_Wi-FiO777U3.pcapng
Packets: 348624 · Displayed: 7099 (2.0%)
Profile: Default

Aim:

To design an Extended Star Topology using 3 switches and 15 systems in Cisco Packet Tracer and verify connectivity.

Procedure

1. Open Cisco Packet Tracer
2. Place 3 switches and 15 PCs
3. Connect 5 PCs to each switch
4. Connect the three switches together
5. Assign IP Addresses to all 15 PCs
6. Configure the subnet mask as 255.255.255.0
7. Check that all links are active
8. Use the ping command between PCs to verify connectivity

Sample Output:

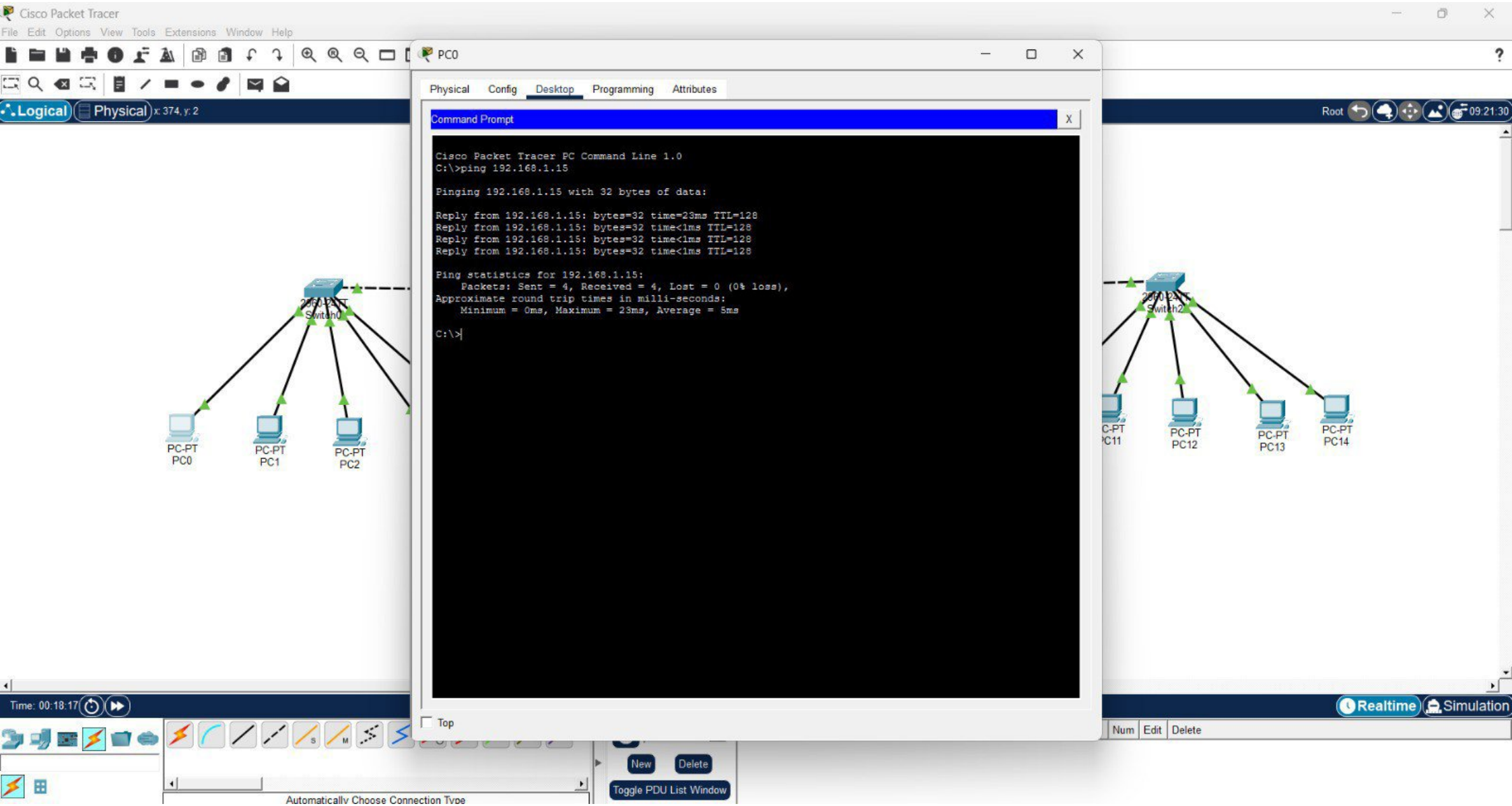
```
PC1 > ping 192.168.1.15
Pinging 192.168.1.15

Reply from 192.168.1.15
Reply from 192.168.1.15
Reply from 192.168.1.15
Reply from 192.168.1.15

Ping successful
```

Result:

Thus, an Extended Star Topology with 3 switches and 15 systems was succesfully designed and connectivity was verified in Cisco Packet tracer


Cisco Packet Tracer
File Edit Options View Tools Extensions Window Help
Logical Physical x: 374, y: 2
Root 09:21:30
PC0
Physical Config Desktop Programming Attributes
Command Prompt
X
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.1.15
Pinging 192.168.1.15 with 32 bytes of data:
Reply from 192.168.1.15: bytes=32 time=23ms TTL=128
Reply from 192.168.1.15: bytes=32 time<1ms TTL=128
Reply from 192.168.1.15: bytes=32 time<1ms TTL=128
Reply from 192.168.1.15: bytes=32 time<1ms TTL=128
Ping statistics for 192.168.1.15:
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 0ms, Maximum = 23ms, Average = 5ms
C:\>
Top
Switch0
PC-PT PC0
PC-PT PC1
PC-PT PC2
Switch2
PC-PT PC11
PC-PT PC12
PC-PT PC13
PC-PT PC14
Time: 00:18:17
Realtime Simulation
Num Edit Delete
New Delete
Toggle PDU List Window
Automatically Choose Connection Type